

Projet Système

Compte rendu du Projet

Table des matières

Introduction.....	1
1. Origine et descriptif du projet.....	1
2. Matériel à disposition	1
I. Déroulement du projet.....	1
1. Séance 1 – 17/03/2025 : Préparation des outils fondamentaux	1
2. Séance 2 – 31/03/2025 : Construction du Shell de base	2
3. Séance 3 – 07/04/2025 : Exécution des commandes et gestion dynamique.....	2
4. Séance 4 – 28/04/2025 : Redirections et pipes	2
5. Séance 5 – 02/05/2025 : Exécution en background et foreground :	3
III. Difficultés rencontrées et solutions apportées	3
V. Répartition du travail.....	3
VI. Conclusion.....	4

Driss HAMADOU

JULIEN TAP

Issam CHAFIQ

Polytech Paris-Saclay

ET3 IIM : Système d'exploitation

Introduction

1. Origine et descriptif du projet

Dans le cadre de la matière "Systèmes d'exploitation" de la formation d'ingénieur à Polytech, nous sommes amenés à concevoir un Shell Unix simplifié en langage C.

Ce projet, réalisé en trinôme, a pour objectif de nous familiariser de manière approfondie avec les mécanismes fondamentaux du système d'exploitation Unix/Linux, à travers l'implémentation de fonctionnalités essentielles d'un terminal.

Le cahier des charges imposait la mise en œuvre progressive des fonctionnalités suivantes :

- L'exécution de programmes dans des processus enfants à partir de commandes tapées dans le terminal.
- Le support de la commande `cd` pour naviguer dans les répertoires. La redirection de l'entrée (`<`) et de la sortie (`>`), ainsi que leur combinaison.
- La gestion des pipes simples et multiples entre commandes.
- L'exécution en arrière-plan (`&`) sans bloquer le Shell.
- La finalisation et l'intégration du gestionnaire de fichier réalisé en TP noté.

Nous avons structuré notre travail en cinq séances principales de développement, chacune dédiée à l'implémentation de blocs fonctionnels précis.

2. Matériel à disposition

Pour réaliser ce projet, nous avons eu à notre disposition :

- Trois ordinateurs portables équipés d'une connexion internet et de l'IDE VSCode.
 - Les bibliothèques standards
 - Le logiciel CMake pour la compilation du projet codé en C.
 - Le logiciel Git et son gestionnaire GitHub pour la gestion collaborative du code.
- I. Déroulement du projet

Le développement du Shell s'est déroulé sur cinq séances de TP, chacune orientée autour d'un ensemble fonctionnel bien défini. Ce découpage nous a permis d'adopter une démarche incrémentale, avec des tests réguliers à chaque étape pour garantir la stabilité et la cohérence du code.

I. Déroulement du projet

1. Séance 1 – 17/03/2025 : Préparation des outils fondamentaux

Cette première séance a été consacrée à un travail de remise à niveau sur le langage C, avec un accent particulier sur :

- La manipulation fine des pointeurs.
- L'allocation et la libération de mémoire dynamique, et les structures de données comme les tableaux dynamiques.

Ces concepts ont été directement réutilisés par la suite dans notre implémentation d'un parseur et du gestionnaire d'arguments de commandes.

Cette phase a aussi permis d'installer un environnement de travail commun et de démarrer la prise en main de Git avec la création du dépôt Git.

2. [Séance 2 – 31/03/2025 : Construction du Shell de base](#)

Nous avons ensuite développé le noyau initial du Shell dans main.c, capable :

- D'afficher un prompt personnalisé.
- De lire une ligne de commande depuis l'entrée standard.
- D'interpréter en appelant un parseur conçu dans input.c.

Le parseur permet de séparer les arguments d'une commande saisie par l'utilisateur, avec un traitement des espaces.

Des tests unitaires ont été menés pour s'assurer que chaque commande était correctement découpée, quel que soit le nombre d'arguments ou la structure syntaxique.

3. [Séance 3 – 07/04/2025 : Exécution des commandes et gestion dynamique](#)

L'objectif principal de cette séance était de permettre l'exécution de commandes standards, en créant un processus enfant avec fork, suivi d'un execvp.

Le fichier systemFunctions.c a été introduit pour regrouper les fonctions système liées à l'exécution.

Nous avons également conçu un module dynamicArray.c, chargé de gérer dynamiquement les tableaux de chaînes de caractères utilisés pour les arguments des commandes.

L'intégration de ce module a amélioré la robustesse et la flexibilité de notre Shell.

Des tests ont permis de valider l'exécution correcte de commandes comme ls, echo, ou encore des exécutables locaux (./nvim).

4. [Séance 4 – 28/04/2025 : Redirections et pipes](#)

Cette étape a marqué un tournant important dans la complexité du Shell.

En effet, deux modules principaux ont été introduits :

- io.c pour la gestion des redirections d'entrée/sortie (<, >) .
- pipe.c pour l'implémentation des pipes entre commandes (|) .

Nous avons d'abord géré les redirections simples, en utilisant les appels systèmes open, dup2, et close, puis combiné les redirections multiples dans des commandes uniques.

Concernant les pipes, la difficulté résidait dans la création de plusieurs processus interconnectés par des canaux de communication. Une gestion rigoureuse des descripteurs et de la fermeture conditionnelle des pipes a été nécessaire pour éviter les blocages.

Des tests successifs ont confirmé la bonne exécution de chaînes de commandes comme :
cat fichier.txt | grep erreur | sort > resultat.txt

5. Séance 5 – 02/05/2025 : Exécution en background et foreground :

Lors de la dernière séance, nous avons implémenté la gestion des commandes en arrière-plan (&) en utilisant `waitpid(WNOHANG)` pour éviter le blocage du shell. Le système de jobs a été adapté pour gérer à la fois les processus foreground (attente active) et background (exécution parallèle).

Les principales modifications incluent :

- Mise à jour des fonctions `put_job_in_foreground/background()`
- Ajout d'un mécanisme de notification des changements d'état
- Intégration d'une bannière au démarrage
- Tests complets des interactions pipes/redirections

Le shell est maintenant fonctionnel avec une gestion complète des processus conforme aux standards Unix.

III. Difficultés rencontrées et solutions apportées

- **Gestion mémoire** : Fuites et erreurs de segmentation → Tableau dynamique implémenté (`dynamicArray.c`)
 - **Redirections** : Problèmes avec `dup2/open` → Module `io.c` créé pour centraliser la logique
 - **Pipes multiples** : Blocages possibles → Approche pas-à-pas avec gestion stricte des descripteurs
 - **Background** : La gestion de la reprise du terminal après avoir mis un job en background
 - **Git** : Conflits de fusion → Processus clair établi (pull avant push, commits atomiques)
 - **CMake** : Configuration complexe → Structure standardisée (`src/` pour le code, `include/` pour les headers)

V. Répartition du travail

Dès le début du projet, nous avons fait le choix de travailler ensemble sur l'ensemble des fonctionnalités, sans découper strictement le travail en blocs indépendants. Cette approche collaborative avait pour objectif de permettre à chacun d'approfondir sa compréhension de tous les aspects du développement d'un Shell, et de favoriser une montée en compétences collective.

Néanmoins, au fil du projet, des spécialisations naturelles se sont formées en fonction des affinités et de la prise d'initiative de chacun :

- Driss HAMADOU s'est concentré sur la structure générale du projet, notamment l'implémentation du fichier `main.c`, la gestion des entrées utilisateur (`input.c`), les fonctions système liées à l'exécution des commandes (`systemFunctions.c`), ainsi que l'intégration du travail du TP noté sur la gestion de fichiers. Il a également mis en place l'architecture CMake, facilitant la compilation modulaire du projet.
- Julien TAP s'est particulièrement investi dans la création et l'optimisation de la structure de tableau dynamique (`dynamicArray.c`) permettant de stocker les arguments de façon souple. Il a

également été moteur dans l'implémentation de la gestion des processus en arrière-plan, garantissant un fonctionnement fluide du Shell lors de l'exécution parallèle de commandes.

- Issam CHAFIQ a pris en charge la partie relative aux entrées/sorties (io.c) avec redirections complexes, ainsi que la mise en place des pipes simples et multiples (pipe.c), assurant une communication correcte entre processus dans des cas de chaînes de commandes.

Tout au long du développement, nous avons procédé à des revues de code régulières, et chaque fonctionnalité a été testée et validée collectivement. Même si certains membres ont initié ou approfondi certaines parties, tous ont contribué aux réflexions, à la correction de bugs, et à l'amélioration du code sur l'ensemble du projet.

VI. Conclusion

Ce projet a été une expérience riche en apprentissages techniques et méthodologiques. Nous avons approfondi notre maîtrise du développement système en C, notamment la gestion des processus, les entrées/sorties et les communications inter-processus via les pipes. Au-delà de la programmation pure, nous avons acquis une première expérience concrète avec des outils professionnels comme Git pour le versionnage collaboratif et CMake pour la gestion de la compilation.

Malgré les défis techniques rencontrés (redirections complexes, gestion des processus en arrière-plan), une organisation rigoureuse et une bonne coordination d'équipe nous ont permis de développer un shell pleinement fonctionnel supportant les commandes de base, les pipes et les redirections. Ce projet constitue une base solide pour d'éventuelles extensions comme l'historique des commandes ou l'autocomplétion.

Au final, cette expérience nous a donné une compréhension plus intime du fonctionnement des shells Unix tout en renforçant nos compétences en développement C et en travail collaboratif. Un projet formateur qui marque une étape importante dans notre parcours en informatique système.